

PRESENTATION ON

30MAL

Full-Stack Maintenance Platform

ASP.NET Core MVC

Entity Framework

C#

December 2025

Problem Statement

Introducing 3OMAL

THE PROBLEM

Fragmented Maintenance Processes

Manual processes lead to significant delays in service requests.

Communication Breakdown

Disorganized communication between customers, technicians, and administrators.

Lack of Transparency

No real-time tracking or status updates for service requests.

User Frustration

Cumbersome experience leading to dissatisfaction.

THE 3OMAL SOLUTION

Streamlined Solution

Centralized platform for efficient maintenance management.

App-Driven Experience

Seamless service booking and tracking through an intuitive interface.

Structured Interaction

Clear workflow between customers, technicians, and administrators.

Efficient Service Delivery

Transforming disorganized processes into a seamless experience.

Project Objectives

FUNCTIONAL OBJECTIVES

- 1

User Registration & Authentication
Secure user and technician management
- 2

Service Booking & Request Management
Service request tracking and management
- 3

Technician Assignment & Dispatch
Efficient request allocation to technicians
- 4

Payment Processing
Secure payment integration
- 5

Feedback & Rating System
User reviews and feedback collection

NON-FUNCTIONAL OBJECTIVES

- Security**
Data privacy and protection
- Performance**
Fast response times
- Scalability**
Support growing user base
- Usability**
User-friendly interface
- Reliability**
High availability

High-Level System Overview

3OMAL orchestrates interactions between customers, technicians, and administrators for efficient maintenance service delivery.

1

Customer Experience

Streamlined service requests with real-time status updates and transparent communication.

2

Technician Efficiency

Optimized assignment and scheduling with clear instructions and location tracking.

3

Administrative Control

Comprehensive dashboard for monitoring, managing resources, and analyzing service trends.

Technology Stack

1

C#

Primary programming language for business logic

2

ASP.NET Core MVC

Framework for building backend APIs

3

SQL Server

Relational database for data storage

4

Bootstrap

CSS framework for UI design

5

jQuery

JavaScript library for client-side interactivity

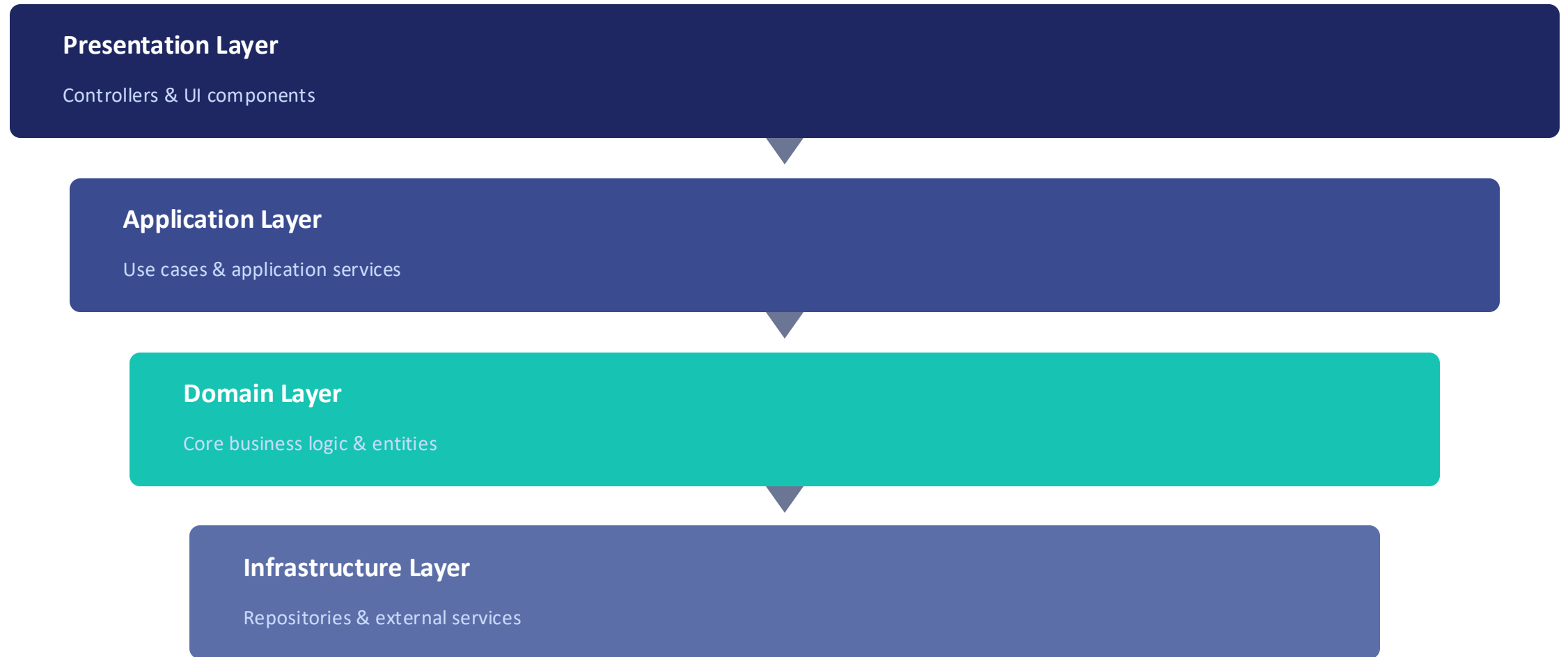
6

Entity Framework

ORM for database interactions

Architectural Flow

Clean architecture organized into four distinct layers



Database Design (ERD)

Data Integrity

1

Foreign Key Constraints

Maintains referential integrity across related entities.

2

Normalization

Reduces data redundancy throughout the schema.

3

Indexing

Optimizes query performance for dashboard analytics.

12+ entities model users, tasks, payments, ratings, notifications, and technician schedules — built for scale and analytics.

OOP and SOLID Principles

S

Single Responsibility Principle

Each class has one reason to change

O

Open/Closed Principle

Open for extension, closed for modification

L

Liskov Substitution Principle

Subtypes must be substitutable for base types

D

Dependency Inversion Principle

High-level modules should not depend on low-level modules



```
// BookingService handles business logic
public class BookingService : IBookingService
{
    private readonly IBookingRepository _repo;

    // Constructor injection for dependency
    public BookingService(IBookingRepository repo)
    {
        _repo = repo;
    }

    // Single responsibility: business logic only
    public async Task<bool> CreateBooking(
        BookingDto dto)
    {
        var booking = new Booking { /* map */ };
        return await _repo.AddAsync(booking);
    }
}
```


Potential Challenges



Real-time Technician Management

Ensuring accurate updates on technician availability and efficient dispatching.

MITIGATION

Implement real-time communication (WebSockets via SignalR) and scheduling algorithms.



Data Security & Privacy

Handling sensitive data requires stringent security measures to protect against breaches.

MITIGATION

Employ encryption, implement strong authentication, and conduct regular security audits.



Platform Scalability

Maintaining performance as user base and service requests grow.

MITIGATION

Design for scalability using cloud-native services, efficient indexing, and caching.



System Reliability

Ensuring high availability and minimizing downtime that impacts user satisfaction.

MITIGATION

Implement monitoring, robust error handling, and design for high availability.

Summary and Deliverables

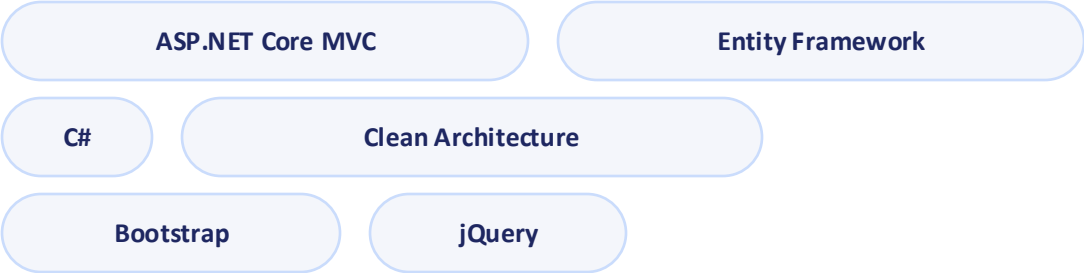
3OMAL successfully delivers a robust full-stack maintenance platform, showcasing proficiency in modern development practices and technologies.

KEY DELIVERABLES

- ✓ User-friendly interface for service booking and management
- ✓ Streamlined service request workflow from booking to completion
- ✓ Scalable backend infrastructure with clean architecture
- ✓ Implementation of SOLID principles for maintainable code
- ✓ Comprehensive admin dashboard for platform management

TECHNICAL IMPLEMENTATION

Technologies Used



Secure user authentication and authorization

- **Key Technical Features**
- Layered architecture ensuring separation of concerns
- Dependency injection for loose coupling
- Comprehensive testing strategy for reliability

Thank You!

Thank you for your attention. The 3OMAL project demonstrates proficiency in ASP.NET Core MVC and Clean Architecture.

User-friendly Interface

Streamlined Service Booking

Scalable Backend

Questions & Answers

- ? Technical architecture and design decisions
- ? Project challenges and enhancements
- ? Feel free to ask about implementation aspects